

CH08 数据库和缓存

考试复习笔记

来源：开源操作系统课件（56 页） 日期：2026 年 6 月 23 日

1 生产环境中的数据库 [p.5–12]

1.1 开发环境 vs 生产环境 [p.5]

维度	开发环境	生产环境
连接方式	本地 Socket/localhost	网络连接，需考虑延迟
监听地址	通常 127.0.0.1	特定 IP 或 0.0.0.0
权限管理	root 直接操作	最小权限原则，多账号分离
连接数量	少量连接	高并发，连接池必备
数据安全	无所谓	备份恢复、容灾

1.2 三种部署结构 [p.6]

- 1. **All-in-One**: 单台服务器包含 Nginx + Web 应用 + MySQL
- 2. **应用与数据库分离**: 应用服务器 + 数据库服务器（内网连接）
- 3. **云数据库 RDS**: 应用服务器 → 云数据库（内网连接）

1.3 数据库监听地址 [p.7]

配置	适用场景
127.0.0.1:3306	仅本机回环（开发环境）
0.0.0.0:3306	所有网络接口（风险高）
10.0.0.5:3306	特定内网 IP（ 生产推荐 ）

1.4 账号权限分离 [p.8]

应用账号（仅业务库 CRUD）、只读账号（报表）、备份账号（SELECT + LOCK TABLES）、管理账号（DBA 全权限）。

1.5 数据库连接池 [p.9–10]

原理：预先建立并维护一批连接，需要时获取，用完归还。

- 太小：请求等待 → 响应时间上升 → 高峰期超时
- 太大：数据库连接占满 → 线程/内存压力增加 → 慢查询长期占用连接

Spring Boot 默认使用 **HikariCP** 连接池。

1.6 慢查询 [p.11–12]

开发环境快但生产变慢的原因：数据量增长、缺少索引、排序分页成本升高、并发请求变多、锁等待增多。

分析方法：分析执行计划 (EXPLAIN)、检查索引使用、优化 SQL 语句。

2 Web 应用缓存技术 [p.14–19]

2.1 多层缓存架构 [p.15]

层次	速度	容量	共享范围	场景
浏览器缓存	最快	小	单用户	CSS/JS/图片
CDN 缓存	快	大	所有用户	静态资源
Nginx 缓存	快	中	所有用户	API 响应
本地缓存	快	小	单实例	热点数据
分布式缓存	中	大	多实例共享	会话、共享数据

2.2 本地缓存方案 [p.16]

ConcurrentHashMap (JDK 原生) → Guava Cache (Google) → **Caffeine** (高性能, Spring Boot 默认) → Ehcache (老牌)。

2.3 本地缓存局限 [p.19]

重启丢失、多实例不一致、受 JVM 堆内存限制。

3 分布式缓存技术与 Redis [p.21–40]

3.1 Memcached vs Redis [p.22–23]

维度	Memcached	Redis
数据结构	仅 String	String/Hash/List/Set/ZSet
持久化	不支持	RDB/AOF
集群	客户端分片	原生 Cluster 支持
线程	多线程	单线程 (6.0+ 多线程 IO)

3.2 Redis 五种核心数据结构 [p.31–39]

类型	操作	典型场景
String	SET/GET/INCR/DECR	访问量统计、缓存
Hash	HSET/HGET/HGETALL/HINCRBY	用户信息对象
List	LPUSH/RPUSH/LPOP/RPOP/LRANGE	消息队列
Set	SADD/SMEMBERS/SINTER/SUNION/SDIFF	标签系统
Sorted Set	ZADD/ZRANGE/ZREVRANGE/ZRANK	排行榜

3.3 Redis 持久化 [p.40]

方式	原理	优缺点
RDB	定时生成数据快照	文件紧凑恢复快，可能丢失最后快照后数据
AOF	记录每次写操作命令	数据更安全，文件较大恢复较慢

AOF 同步策略：everysec（折中）/ always（最安全但慢）/ no（最快但可能丢数据）。

4 缓存核心问题与解决方案 [p.42–50]（高频考点）

4.1 缓存穿透 [p.42]

问题：大量请求查询缓存和数据库中都不存在的数据。

解决：参数校验、空值缓存、布隆过滤器、限流、鉴权和频率控制。

4.2 缓存击穿 [p.44]

问题：热点 key 过期瞬间，大量请求同时打到数据库。

解决：更长过期时间、逻辑过期（先返旧值后台刷新）、互斥锁（只允许一个请求重建缓存）、热点数据预热、数据库限流。

4.3 缓存雪崩 [p.46] (Redis 崩溃相关核心考点)

问题：大量缓存同时失效，或缓存服务整体不可用。

常见原因：

- 大量 key 使用相同过期时间
- **Redis 宕机**
- 发布时清空缓存
- 缓存集群故障
- 网络故障导致应用访问不了 Redis

解决方案：

1. 过期时间加入随机抖动
2. 启动前缓存预热
3. **Redis 高可用**（主从 + Sentinel）
4. 应用限流和降级
5. 多级缓存
6. 保留数据库保护阈值

4.4 缓存一致性 [p.48]

问题：数据库已更新，缓存仍是旧值。

方案：Cache Aside 模式、延迟双删策略。取决于业务要求选择。

5 综合案例：电商商品详情页缓存设计 [p.52–56]

5.1 场景 [p.52]

电商商品详情页，QPS 10000，读多写少。

5.2 多层缓存策略 [p.53]

数据	存储位置	TTL	说明
商品基本信息	Redis + 本地缓存	10min / 1min	频繁访问
商品价格	Redis	5min	需要及时更新
商品库存	不缓存	实时查询	一致性要求高
商品评价	Redis List	30min	允许短暂不一致

5.3 关键问题 [p.54–56]

1. 价格变动如何更新缓存？：使用 Cache Aside 或延迟双删策略
2. 如何防止大促时缓存雪崩？：过期时间加随机抖动、缓存预热、Redis 高可用、多级缓存、限流降级

开源操作系统 · CH08 数据库和缓存 · 56 页课件全覆盖